

Programación para Automatización 1

TPG2 : Trabajo Práctico Grupal N.º 2

Modalidad

Grupal (hasta 7 integrantes)

Objetivos

Aplicar los conceptos aprendidos sobre:

- FastAPI
- Uvicorn
- APIs REST
- Métodos HTTP (GET, POST, PUT, DELETE)
- Pydantic
- Validación de datos
- Bearer Token
- Persistencia en archivos JSON
- Consumo de APIs con Requests
- Modularización del código
- Manejo de errores

Situación problemática

Un hospital necesita informatizar la gestión de:

- Pacientes
- Médicos
- Internaciones

Inicialmente no se utilizará una base de datos. Toda la información deberá almacenarse en archivos JSON.

El sistema estará compuesto por dos partes:

1. Una API REST desarrollada con FastAPI.
2. Una aplicación cliente en Python que consumirá dicha API.

Parte 1 – Desarrollo de la API REST

Requisitos generales

La API deberá:

- Ejecutarse mediante Uvicorn.
- Utilizar modelos Pydantic.
- Validar los datos recibidos.
- Implementar operaciones CRUD.
- Persistir la información en archivos JSON.
- Proteger rutas mediante Bearer Token.

Recurso 1: Pacientes

Modelo sugerido	Endpoints sugeridos		
<pre>class Paciente(BaseModel): id: int dni: str nombre: str apellido: str edad: int telefono: str obra_social: str</pre> Archivo de persistencia: pacientes.json	Método	Ruta	Descripción
	POST	/pacientes	Crear paciente
	GET	/pacientes	Listar pacientes
	GET	/pacientes/{id}	Buscar paciente
	PUT	/pacientes/{id}	Modificar paciente
	DELETE	/pacientes/{id}	Eliminar paciente

Recurso 2: Médicos

Modelo sugerido	Endpoints sugeridos												
<pre>class Medico(BaseModel): id: int matricula: str nombre: str apellido: str especialidad: str telefono: str</pre> Archivo de persistencia: medicos.json	<table> <thead> <tr> <th>Método</th><th>Ruta</th></tr> </thead> <tbody> <tr> <td>POST</td><td>/medicos</td></tr> <tr> <td>GET</td><td>/medicos</td></tr> <tr> <td>GET</td><td>/medicos/{id}</td></tr> <tr> <td>PUT</td><td>/medicos/{id}</td></tr> <tr> <td>DELETE</td><td>/medicos/{id}</td></tr> </tbody> </table>	Método	Ruta	POST	/medicos	GET	/medicos	GET	/medicos/{id}	PUT	/medicos/{id}	DELETE	/medicos/{id}
Método	Ruta												
POST	/medicos												
GET	/medicos												
GET	/medicos/{id}												
PUT	/medicos/{id}												
DELETE	/medicos/{id}												

Recurso 3: Internaciones

Modelo sugerido	Endpoints sugeridos
<pre>class Internacion(BaseModel): id: int paciente_id: int medico_id: int fecha_ingreso: str diagnostico: str habitacion: int estado: str</pre> Archivo de persistencia: internaciones.json	Método Ruta POST /internaciones GET /internaciones GET /internaciones/{id} PUT /internaciones/{id} DELETE /internaciones/{id}

Persistencia en JSON

La información deberá mantenerse aunque la API sea reiniciada.

Se recomienda implementar funciones similares a:

```
def cargar_pacientes():
```

```
def guardar_pacientes():
```

Seguridad mediante Bearer Token

Rutas protegidas

Se recomienda proteger:

Método

- POST
- PUT
- DELETE

Validaciones mínimas

Pacientes	Médicos	Internaciones
• edad > 0 • nombre obligatorio • apellido obligatorio	• matrícula obligatoria • especialidad obligatoria	• paciente_id existente • medico_id existente • habitación positiva

Parte 2 – Desarrollo de la Aplicación Cliente

Objetivo

Desarrollar una aplicación de consola en Python para uso del personal administrativo y médico.

La aplicación deberá consumir la API desarrollada en la Parte 1.

Librerías sugeridas

- requests
- json

Menú principal

Ejemplo:

```
=====
SISTEMA HOSPITALARIO
=====
```

- 1 - Gestionar pacientes
- 2 - Gestionar médicos
- 3 - Gestionar internaciones
- 4 - Salir

Módulo de Pacientes

Opciones

- 1 - Alta paciente
- 2 - Buscar paciente
- 3 - Modificar paciente
- 4 - Eliminar paciente
- 5 - Listar pacientes

Consumo esperado

Alta

```
requests.post(...)
```

Listado

```
requests.get(...)
```

Modificación

```
requests.put(...)
```

Eliminación

`requests.delete(...)`

Módulo de Médicos

Opciones similares.

Módulo de Internaciones

Opciones similares.

Requisitos técnicos

La aplicación cliente deberá:

- Utilizar funciones.
- Validar entradas del usuario.
- Manejar errores con try/except.
- Mostrar mensajes amigables.
- Utilizar menús de navegación.
- Consumir correctamente la API.

Funcionalidad adicional

Generar archivo de log: logs.txt

registrando:

- fecha
- usuario
- operación realizada

Presentar

Cada grupo deberá presentar:

- **Código fuente de la API**
- **Código fuente de la aplicación cliente**
- **Archivos JSON**
 - pacientes.json
 - medicos.json
 - internaciones.json
- **Archivo de Integrantes**